
Algorithms and Computability - Project Report

Czajkowska Olga, Leymańczyk Magdalena, Pietraś Jakub, Pleszyńska Hanna

Warsaw, 05.12.2024

1 Graph size

We define **graph size** $|G|$ as the cardinality of the set of all edges E of a graph $G = (V, E)$, denoted as $|E|$. Given the adjacency matrix $A \in \mathbb{N}^{n \times n}$, where $n = |V|$ is the number of vertices, and $A[i][j]$ represents the number of edges between vertices i and j , the size of G is calculated as follows:

- For **undirected graphs**

$$|G| = |E| = \frac{1}{2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} A[i][j]$$

- For **directed graphs**

$$|G| = |E| = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} A[i][j]$$

2 Graph Edit Distance

Graph edit distance (GED) is a measure of similarity between two graphs, defined as the minimum cost of transforming one graph into another through a series of edit operations: node and edge insertion, deletion, or substitution. This section introduces the necessary definitions, the concept of GED as a metric, describes a process of its computation, as well as proposes a method for its fast approximation.

2.1 GED Definition

To fully define Graph Edit Distance, it is necessary to first introduce key concepts and definitions. We assume $G_1 = (V_1, E_1)$ to be a graph that is transformed into graph $G_2 = (V_2, E_2)$.

- **Node map** is a partial or a complete function that assigns each node $u \in V_1$ to a node $v \in V_2$, or indicates deletion of a node. It is denoted as $M = \{e_1, \dots, e_{|M|}\}$, where e_i is the i -th edit operation.
- **Edit path** is a sequence of edit operations (insertions, deletions, substitutions) that transforms G_1 into G_2 completely. The set of all possible edit paths between G_1 and G_2 is denoted as $\Upsilon(G_1, G_2)$.

- **Cost**, denoted as $\mathcal{C}(e)$, is a function which measures the "strength" of a corresponding operation e . The idea of such a cost is to define whether or not an edit operation represents a strong modification of the graph.[1]

Consequently, the edit distance of the two graphs is defined by the minimum cost edit path between two graphs:

$$GED(G_1, G_2, \mathcal{C}) = \min_{(e_1, \dots, e_k) \in \Upsilon(G_1, G_2)} \sum_{i=1}^k \mathcal{C}(e_i)$$

2.2 Metric

A **metric** d on a set X is a function $d : X \times X \rightarrow \mathbb{R}$ such that for all $x, y \in X$:

1. $d(x, y) \geq 0$ and $d(x, y) = 0$ if and only if $x = y$;
2. $d(x, y) = d(y, x)$ (*symmetry*);
3. $d(x, y) \leq d(x, z) + d(z, y)$ (*triangle inequality*).

The concept of a metric allows us to introduce the notion of a distance in the set of all graphs.

2.3 GED as a metric

In general, graph edit distance is not necessarily a metric [4]. To guarantee that we obtain a metric, the cost function needs to be properly defined according to a set of conditions.

1. $\mathcal{C}(e) \geq 0$ for all node and edge operations e (*Non-negativity*)
2. $\mathcal{C}(e) = 0$ if and only if the operation e is an identical node or edge substitution (*identity of indiscernibles*)
3. $\mathcal{C}(e) = \mathcal{C}(e^{-1})$ for any edit operation e on nodes and edges, where e^{-1} denotes the inverse edit operation to e (*symmetry*)
4. For all nodes u, v and w and corresponding operations, the following *triangle inequality* holds

$$\begin{aligned} \mathcal{C}(u \rightarrow w) &\leq \mathcal{C}(u \rightarrow v) + \mathcal{C}(v \rightarrow w) \\ \mathcal{C}(u \rightarrow \varepsilon) &\leq \mathcal{C}(u \rightarrow v) + \mathcal{C}(v \rightarrow \varepsilon) \\ \mathcal{C}(\varepsilon \rightarrow v) &\leq \mathcal{C}(\varepsilon \rightarrow u) + \mathcal{C}(u \rightarrow v) \end{aligned}$$

The set of all possible edit paths $\Upsilon(G_1, G_2)$ is infinite, because we can include an arbitrary number of unnecessary substitutions, or insertions followed by deletions of the same node. However, conditions 1., 2. and 4. ensure that the search for the minimal cost path is limited to a finite set.

Specifically, condition 1. guarantees that redundant operations like repeated substitutions or superfluous edits will always increase or maintain the total cost. Condition 2. ensures that paths with unnecessary operations, such as inserting and then immediately deleting the

same node or edge, result in a strictly positive cost. These conditions make such superfluous paths suboptimal, and they can therefore be excluded from the search. Consequently, only paths that contain the minimum necessary edits are considered, which limits the search space to a finite subset of $\Upsilon(G_1, G_2)$.

The triangle inequality ensures that the direct cost of substituting u with w ($c(u \rightarrow w)$) is always less than or equal to the cost of substituting u with v and then v with w ($c(u \rightarrow v) + c(v \rightarrow w)$). This guarantees that no minimal-cost edit path contains unnecessary intermediate substitutions, as such paths can always be replaced by shorter ones with equal or lower total cost.

Lastly, condition 3. ensures that the cost of transforming graph G_1 into G_2 is equal to the cost of transforming G_2 into G_1 , i.e., $\text{GED}(G_1, G_2) = \text{GED}(G_2, G_1)$. This property guarantees that the edit distance is independent of the transformation direction.

2.4 Graph Edit Distance - Exact Computation

Graph Edit Distance belongs to the family of quadratic assignment problems (QAP), which belong to the class of NP-complete problems. Therefore exact computation of graph edit distance can be solved in exponential time complexity only. [1] Our solution utilizes a brute force algorithm, which generates all possible complete edit paths, computes the edit cost for each, and determines the smallest such value.

The general steps of this exhaustive search are detailed in Algorithm 1. Some of the key elements of this solution are detailed below.

1. To simplify the calculations and ensure that GED is a metric, we use the following cost function:

$$\mathcal{C}(e) = \begin{cases} 1 & \text{if } e \text{ is an insertion or deletion of a node or an edge,} \\ 0 & \text{if } e \text{ is a substitution of a node or an edge.} \end{cases}$$

2. A single mapping is a list that holds all possible nodes in G_2 that a given node in G_1 (indicated by the index of the list) can be mapped to. **Example** `mapping[0] = [1 0 -1]` represents three possible operations for $u_0 \in G_1$: $(u_0 \rightarrow v_1)$, $(u_0 \rightarrow v_0)$ and $(u_0 \rightarrow \epsilon)$.
3. The algorithm iterates over all edit paths, checking if they are valid (i.e. no two nodes in G_1 are mapped to the same G_2 node).
4. A mapped node is a node in G_1 or G_2 which is a part of some operation (substitution, deletion or insertion). Consequently, unmapped nodes are those which have not been subjected to any operation.

Algorithm 1 Graph Edit Distance (GED) by Exhaustive Search

```
1: Input:  $G_1, G_2$  (Adjacency matrices)
2: Output: Minimum Graph Edit Distance ( $minCost$ )
3: Initialize  $mappings[i] \leftarrow [v_0, \dots, v_{n_2-1}, -1]$  for each  $u_i \in G_1$ 
4:  $totalMappings \leftarrow \prod_{i=0}^{n_1-1} size(mappings[i])$ ,  $minCost \leftarrow \infty$ 
5: for  $mappingIndex \in [0, totalMappings - 1]$  do
6:   Decode  $mappingIndex \rightarrow currentMapping$ 
7:   Validate  $currentMapping$ : Ensure no two  $u_i \in G_1$  map to the same  $v_j \in G_2$ 
8:   if  $currentMapping$  is invalid then
9:     continue
10:  end if
11:   $totalCost \leftarrow 0$ 
12:  for  $u_i \in G_1$  do
13:    if  $u_i == -1$  then
14:       $totalCost \leftarrow totalCost + 1$  (deletion cost)
15:    end if
16:  end for
17:  for  $v_j \in G_2$  not mapped do
18:     $totalCost \leftarrow totalCost + 1$  (insertion cost)
19:  end for
20:  for edges  $(u_i, u_k) \in G_1$  do
21:    if edge  $(u'_i, u'_k) \in G_2$  between mapped vertices  $u_i$  and  $u_k$  exists then
22:      Compute difference  $D$  of edge multiplicity between  $(u_i, u_k) \in G_1$  and  $(u'_i, u'_k) \in G_2$ 
23:       $totalCost \leftarrow totalCost + D$ 
24:    else
25:      Delete multi-edge  $(u_i, u_k)$ 
26:    end if
27:  end for
28:  for unmapped  $v_j \in G_2$  do
29:    Add edge insertion costs for connections to mapped nodes
30:    Add edge insertion costs for connections between unmapped nodes
31:  end for
32:  if  $totalCost < minCost$  then
33:     $minCost \leftarrow totalCost$ 
34:  end if
35: end for
36: Return  $minCost$ 
```

2.4.1 Complexity analysis

Let us assume that $|G_1| = n$ and $|G_2| = m$. The algorithm enumerates every possible edit path, resulting in $(m + 1)^n$ paths total. For each such possibility, we:

- Validate and decode a path by iterating over n ($\mathcal{O}(n)$)

- Compute node substitutions, deletions and insertions ($\mathcal{O}(n + m)$)
- Consider implied operation costs for edges between each node in G_1 ($\mathcal{O}(n * n)$)
- Consider implied operation costs for edges between unmapped nodes in G_2 ($\mathcal{O}(m * m)$)

The overall time complexity is therefore $\mathcal{O}((m + 1)^n * (n^2 + m^2))$, which grows extremely fast.

2.5 Graph Edit Distance - Approximation

To remedy the high computational complexity of the exact algorithm, we reduce the problem to an instance of a linear sum assignment problem (LSAP). It consists of finding an optimal assignment between two independent sets of entities [1]. Solving the LSAP effectively minimizes the total assignment cost by utilizing algorithms for bipartite graph matching, such as the Hungarian algorithm or its variants.

While very efficient, the main drawback of this approach is its accuracy. Due to the fact that during the node assignment only local information is considered (i.e. no global relationship between nodes is explored), the derived edit distance is at best equal to, and at worst larger than, the exact graph edit distance.

2.5.1 Cost function

As in the exact Graph Edit Distance computation, we use the following cost function:

$$\mathcal{C}(e) = \begin{cases} 1 & \text{if } e \text{ is an insertion or deletion of a node or an edge,} \\ 0 & \text{if } e \text{ is a substitution of a node or an edge.} \end{cases}$$

When considering edit costs of individual node operations, we need to take into account the implied edge operations as well. Those are edge modifications that are necessary when nodes of a graph are modified. To that end, for **undirected graphs** the edit costs are as follows:

- $\mathcal{C}((u_i \rightarrow v_j)) = |deg(u_i) - deg(v_j)|$
- $\mathcal{C}((u_i \rightarrow \epsilon)) = 1 + deg(u_i)$
- $\mathcal{C}((\epsilon \rightarrow v_j)) = 1 + deg(v_j)$

For **undirected graphs** one needs to consider the in and out degrees of nodes. For some node $u \in V$, let us denote them as $id(u)$ and $od(u)$ respectively. Then, the edit costs are:

- $\mathcal{C}((u_i \rightarrow v_j)) = |id(u_i) - id(v_j)| + |od(u_i) - od(v_j)|$
- $\mathcal{C}((u_i \rightarrow \epsilon)) = 1 + id(u_i) + od(u_i)$
- $\mathcal{C}((\epsilon \rightarrow v_j)) = 1 + id(v_j) + od(v_j)$

2.5.2 Construction of a cost matrix

We transform the GED computation problem into LSAP by first constructing the cost matrix $C \in \mathbb{N}^{(n+m) \times (n+m)}$. It represents the edit costs of substitutions, deletions and insertions of nodes, taking into account the implied edge operations costs.

$$C = \begin{array}{c|cccc|cccc} & c_{11} & c_{12} & \cdots & c_{1m} & c_{1\epsilon} & \cdots & \infty \\ & c_{21} & c_{22} & \cdots & c_{2m} & c_{2\epsilon} & \cdots & \infty \\ & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ & c_{n1} & c_{n2} & \cdots & c_{nm} & \infty & \cdots & c_{n\epsilon} \\ \hline & c_{\epsilon 1} & c_{\epsilon 2} & \cdots & c_{\epsilon m} & 0 & \cdots & 0 \\ & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ & \infty & \infty & \cdots & \infty & 0 & \cdots & 0 \end{array}$$

Explanation:

- c_{ij} represents the cost of a node substitution ($u_i \rightarrow v_j$), hence the left upper quadrant of C represents all possible substitution costs
- $c_{i\epsilon}$ represents the cost of a node deletion ($u_j \rightarrow \epsilon$), hence the diagonal in the right upper quadrant of C shows costs of all deletions
- c_{ϵ} represents the cost of a node insertion ($\epsilon \rightarrow v_j$), hence the diagonal in the left lower quadrant of C shows costs of all substitutions

Example Consider undirected graphs:

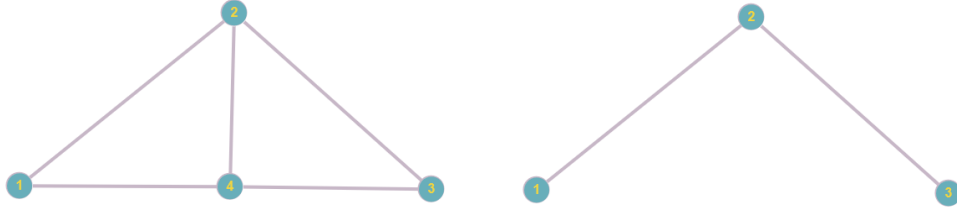


Figure 1: Graphs G_1 (left) and G_2 (right)

The cost matrix for G_1 and G_2 is as follows:

$$\begin{bmatrix} 1 & 0 & 1 & 3 & \infty & \infty & \infty \\ 2 & 1 & 2 & \infty & 4 & \infty & \infty \\ 1 & 0 & 1 & \infty & \infty & 3 & \infty \\ 2 & 1 & 2 & \infty & \infty & \infty & 4 \\ 2 & \infty & \infty & 0 & 0 & 0 & 0 \\ \infty & 3 & \infty & 0 & 0 & 0 & 0 \\ \infty & \infty & 2 & 0 & 0 & 0 & 0 \end{bmatrix}$$

We will look at some computations leading to the creation of C .

1. c_{10} represents a substitution ($u_2 \rightarrow v_1$), and so the edit cost for this operation is $|deg(u_2) - deg(v_1)| = 3 - 1 = 2$
2. c_{14} represents a deletion ($u_2 \rightarrow \epsilon$), having an edit cost of $1 + deg(u_2) = 1 + 3 = 4$
3. c_{40} represents an insertion ($\epsilon \rightarrow v_1$) having an edit cost of $1 + deg(v_1) = 1 + 1 = 2$

2.5.3 Minimal cost computation

The task of solving an instance of LSAP problem on the resultant square matrix corresponds to the task of finding the minimum cost assignment of the nodes of G_1 to the nodes of G_2 [1]. There exist multiple algorithms to solve the LSAP problem that run in polynomial time, most notably the Jonker-Volgenant algorithm and the Hungarian (Munkres) algorithm. Our solution focuses on the latter, and the general steps are detailed in Algorithm 2. The full explanation of the algorithm used is described in [3].

Algorithm 2 Munkres Algorithm

```
1: Input: Cost matrix  $C$ 
2: Output: Optimal assignment with minimum cost
3:
4: Step 0: Prepare Cost Matrix
5: for each row  $i$  in  $C$  do
6:   Subtract the smallest element in row  $i$  from all elements in that row
7: end for
8: Step 1: Star Zeros
9: for each zero  $Z$  in  $C$  do
10:  if no other starred zero exists in the same row or column then
11:    Star  $Z$ 
12:  end if
13: end for
14: Step 2: Cover Columns with Starred Zeros
15: while number of covered columns  $< n$  do
16:   Cover columns containing starred zeros
17:   IF all columns are covered, RETURN optimal assignment
18:   ELSE proceed to Step 3
19: end while
20: Step 3: Prime Zeros and Update Matrix
21: while no uncovered zeros exist do
22:   Prime an uncovered zero  $Z'$ 
23:   if no starred zero exists in the same row as  $Z'$  then
24:     Proceed to Step 4
25:   else
26:     Cover the row of  $Z'$  and uncover the column of the starred zero
27:   end if
28: end while
29: Adjust the cost matrix by subtracting the smallest uncovered value from all uncovered
   elements and adding it to all covered rows. Return to Step 3.
30: Step 4: Augment Paths and Update Stars
31: Construct a path of alternating primed and starred zeros
32: Swap primes and stars along the path
33: Uncover all rows and columns and erase primes
34: Return to Step 2
35: Step 5: Completion
36: When all rows or columns are covered, starred zeros represent the optimal assignments
37: RETURN the assignments and their total cost
```

2.5.4 Complexity analysis

The implementation of the Munkres algorithm used by our program is of a low order polynomial run time, worst-case $\mathcal{O}(n^3)$ (as indicated in [3]).

2.6 Tests of the GED Algorithms

In order to test the accuracy, G_1 was first generated, and then edited manually to form G_2 . This was done to enforce a specific value of GED.

2.6.1 Isomorphic graphs

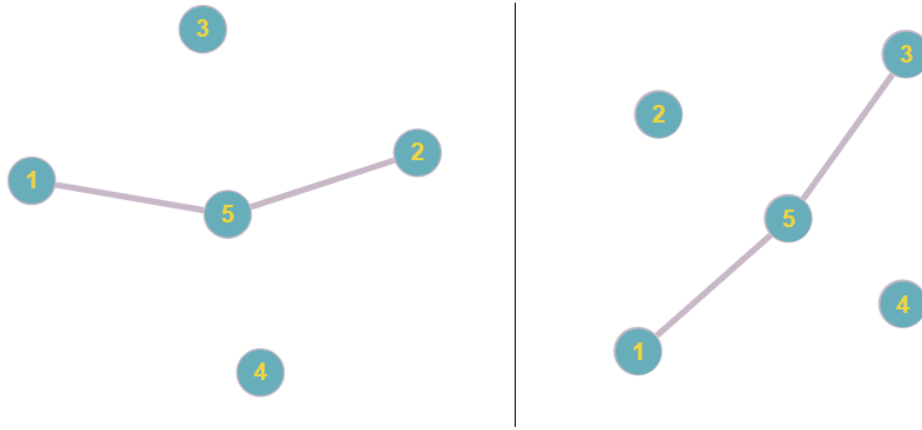


Figure 2: G_1 (left) into G_2 (right). Expected $GED = 0$

	Exact	Approximation
Result	0	0
Execution time	6ms	5ms

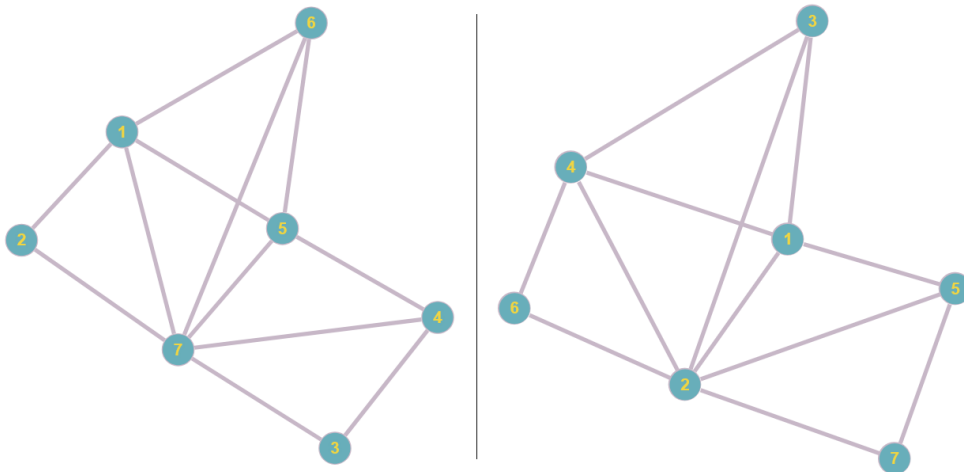


Figure 3: G_1 (left) into G_2 (right). Expected $GED = 0$

	Exact	Approximation
Result	0	0
Execution time	333ms	10ms

2.6.2 Directed multigraph

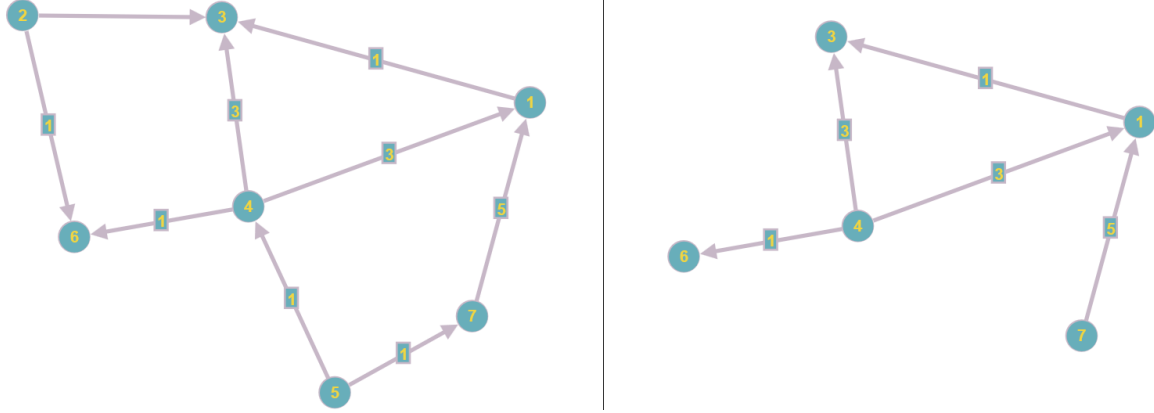


Figure 4: G_1 (left) into G_2 (right). Expected $GED = 6$

	Exact	Approximation
Result	6	10
Execution time	57ms	4ms

2.6.3 Graphs with size approaching the limit for exact algorithm

These tests were carried out to find the maximal size of a graph, for which exact GED is computed within a reasonable time limit (that is **60 seconds**).

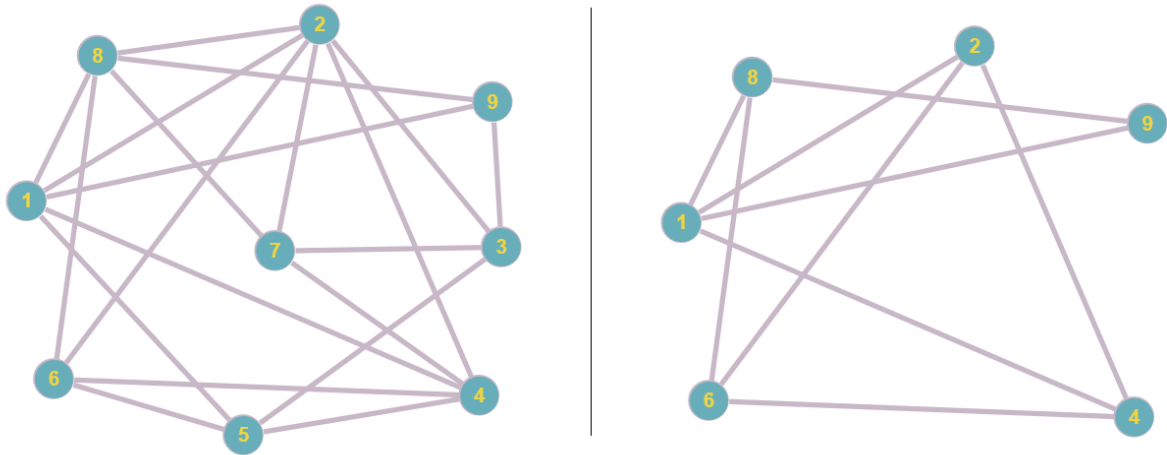


Figure 5: G_1 (left) into G_2 (right). Expected $GED = 14$

	Exact	Approximation
Result	14	25
Execution time	4332ms	8ms

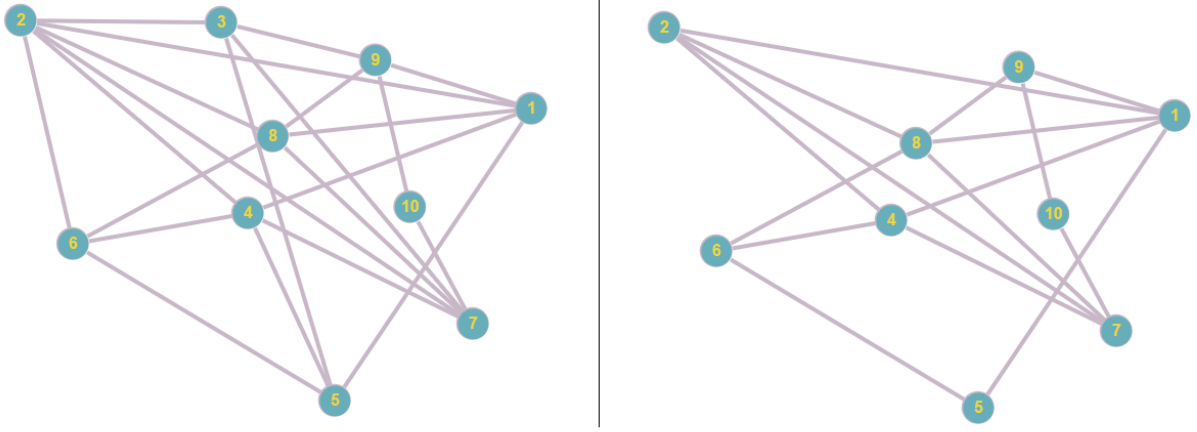


Figure 6: G_1 (left) into G_2 (right). Expected $GED = 7$

	Exact	Approximation
Result	-	13
Execution time	Timeout	41ms

Attempting to run the exact algorithm on graphs with size 10 or more resulted in a timeout. Consequently, the following tests will only focus on the approximation algorithm.

2.6.4 Large graphs

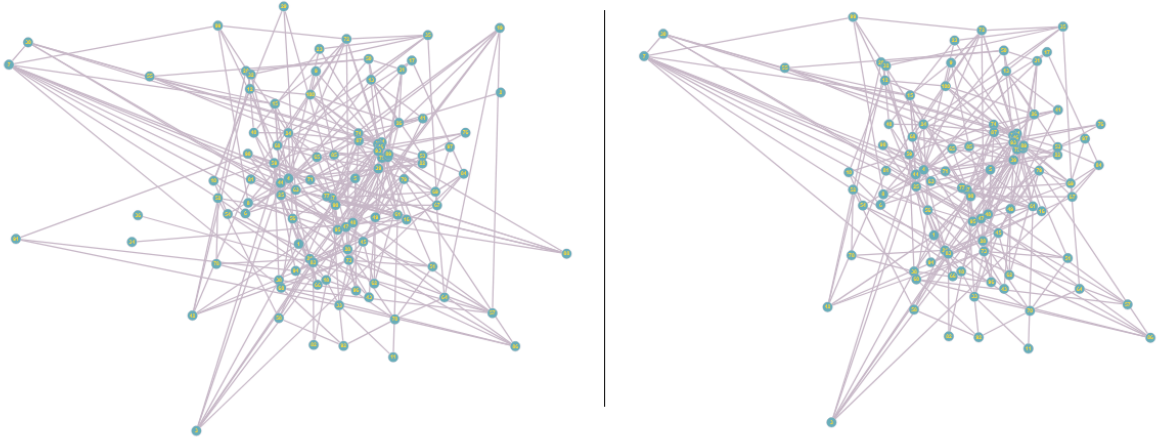


Figure 7: G_1 (left) of size 100 into G_2 (right) of size 93. Expected $GED = 28$

	Approximation
Result	51
Execution time	32ms

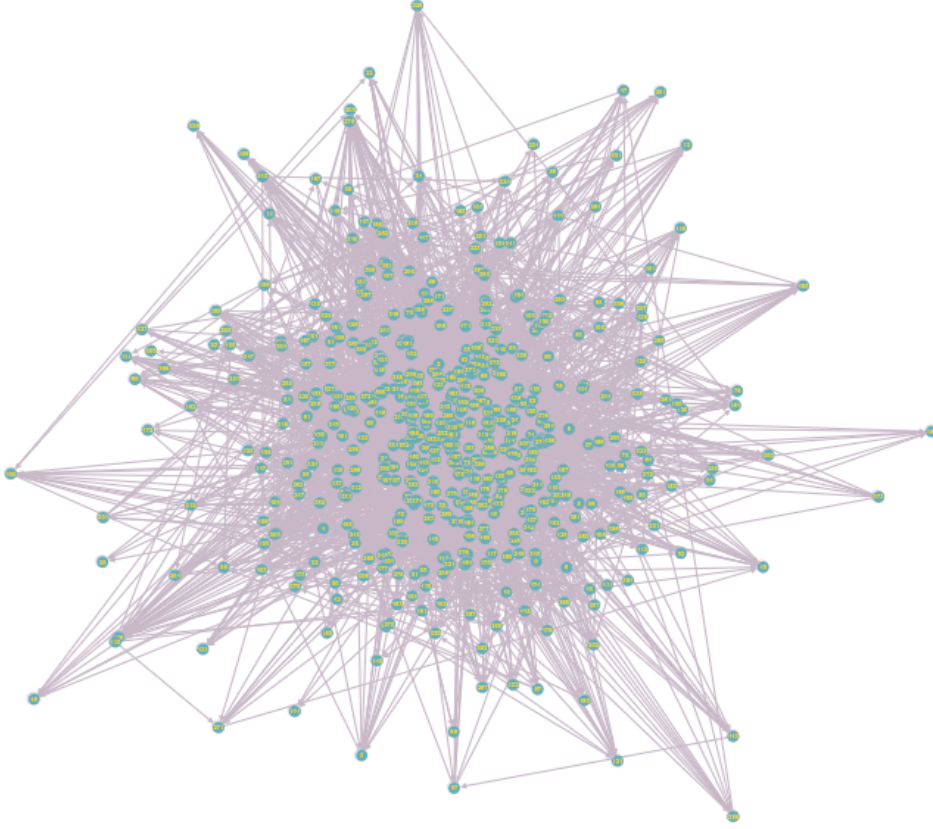


Figure 8: G_1 of size 500. G_2 has been obtained by removing two vertices. Expected $GED = 25$

	Approximation
Result	46
Execution time	2200ms

2.6.5 Tests summary

The results obtained from the exact (brute force) algorithm were consistently accurate for smaller graphs, as it examines every possible mapping. However, the exhaustive nature of this method leads to a rapid increase in time complexity as the graph size grows. Once $|G| \geq 10$, the brute force approach becomes too expensive, making it essentially unusable for larger instances.

On the other hand, the approximation algorithm can handle significantly bigger graphs, even up to 500 vertices, within a reasonable time frame. While this approach does not guarantee finding the minimal edit path, it still delivers useful results efficiently. The trade-off is a reduction in accuracy compared to the exact method, but given the computational difficulties of the brute force solution for larger inputs, it proves to be a worthy compromise.

3 The longest cycle in a graph

A **cycle** is a sequence $v_1, v_2, \dots, v_{k-1}, v_k, v_1$ where $\{v_1, v_2, \dots, v_k\}$ is a set of at least 3 different vertices and each pair of vertices $v_i v_{i+1}$ are connected with at least 1 edge.

The **length of a cycle** is the sum of the number of edges between each pair of subsequent vertices in a cycle.

3.1 Exact algorithm

The exact algorithm for finding the longest cycle is a modification of an algorithm for finding the longest path based on DFS. For each vertex it looks for the cycles containing that vertex by exploring all paths starting in that vertex. When a cycle is found that is bigger than the previous *maxCycle*, *maxCycle* and counter of the longest cycles are adjusted to fit the newest development.

3.1.1 Complexity analysis

Let n be the number of vertices in the multigraph. The function *CyclesFromVertex* calls itself at most $n - 1$ times in one execution, because one vertex can have at most $n - 1$ neighbors. Therefore the time complexity of finding all cycles containing given vertex is $O(n^n)$. This procedure is repeated for every vertex, so the complexity of the whole algorithm is $O(n^{n+1})$.

Algorithm 3 Exact Maximum Cycle

```
1: Input: Multigraph  $G = (V, E)$ 
2: Output: The length and number of the longest paths in  $G$ 
3: Initialize  $maxCycle \leftarrow \emptyset$ 
4: Initialize  $maxCycleCount \leftarrow 0$ 
5: Initialize  $maxLength \leftarrow 0$ 
6: Define recursive function  $CyclesFromVertex(G, v, maxCycle, tempCycle)$ :
7: Append  $v$  to  $tempCycle$ 
8: for all  $w$  connected to  $v$  do
9:   Increase  $tempLength$  by the number of edges between  $v$  and  $w$ 
10:  if  $w \notin tempCycle$  then
11:    Remove all edges  $(v, w)$  from  $G$ 
12:    Call  $CyclesFromVertex(G, w, maxCycle, tempCycle)$ 
13:  else if  $w = tempCycle[0]$  then
14:    if  $tempLength = maxLength$  then
15:      Increase  $maxCycleCount$  by 1
16:    else if  $tempLength > maxLength$  then
17:       $maxCycle \leftarrow tempCycle$ 
18:       $maxLength \leftarrow tempLength$ 
19:    end if
20:  end if
21:  Decrease  $tempLength$  by the number of edges between  $v$  and  $w$ 
22: end for
23: Remove  $v$  from  $tempCycle$ 
24: return
25: for all  $v \in V$  do
26:   Initialize  $tempLength \leftarrow 0$ 
27:   Initialize  $tempCycle \leftarrow \emptyset$ 
28:   Call  $CyclesFromVertex(G, v, maxCycle, tempCycle)$ 
29: end for
30:  $maxCycleCount \leftarrow maxCycleCount / \text{length of } maxPath$ 
31: return  $maxCycleCount, maxLength$ 
```

3.2 Approximation algorithm

To solve the problem in polynomial time, some approximation of the Algorithm 3 needs to be used. We do that by limiting the number of backtracks the program can perform. We introduce a counter that counts how many cycles were found since the last correction of the *maxCycle*. Once the counter exceeds some predefined threshold, the algorithm does not search for more cycles. The algorithm was inspired by [2]

To increase the precision of the approximation for multigraphs the algorithm starts searching for cycles from the neighbor connected to the starting vertex with the highest amount of edges. The time complexity of this algorithm is $O(n^2 * k)$, where $k = \text{maxCorrections}$ because finding the longest cycle takes n iterations and for each time the new longest cycle is found the algorithm can find k cycles that are smaller or equal to the length of *maxCycle*.

Algorithm 4 Approximated Maximum Cycle

```
1: Input: Multigraph  $G = (V, E)$ , int  $maxCorrections$ 
2: Output: The length and number of the longest paths in  $G$ 
3: Initialize  $maxCycle \leftarrow \emptyset$ 
4: Initialize  $maxCycleCount \leftarrow 0$ 
5: Initialize  $maxLen \leftarrow 0$ 
6: Define recursive function  $CyclesFromVertex(G, v, maxCycle, tempCycle, maxLength, tempLength, truncated, maxCorrections)$ :
7: Append  $v$  to  $tempCycle$ 
8: for all  $w$  connected to  $v$  do
9:   Increase  $tempLength$  by the number of edges between  $v$  and  $w$ 
10:  if  $w \notin tempCycle$  then
11:    Remove all edges  $(v, w)$  from  $G$ 
12:    Call  $CyclesFromVertex(G, w, maxCycle, tempCycle)$ 
13:    if  $truncated = \text{True}$  then
14:      return
15:    end if
16:  else
17:    Increase  $foundCyclesNr$  by 1
18:    if  $w = tempCycle[0]$  then
19:      Increase  $foundCyclesNr$  by 1
20:      if  $tempLength = maxLength$  then
21:        Increase  $maxCycleCount$  by 1
22:      else if  $tempLength > maxLength$  then
23:         $maxCycle \leftarrow tempCycle$ 
24:         $maxLength \leftarrow tempLength$ 
25:         $foundCyclesNr \leftarrow 0$ 
26:      end if
27:    end if
28:    if  $foundCyclesNr > maxCorrections$  then
29:       $truncated \leftarrow \text{True}$ 
30:    return
31:    end if
32:  end if
33: end for
34: Remove  $v$  from  $tempCycle$ 
35: return
36: for all  $v \in V$  do
37:   Initialize  $tempCycle \leftarrow \emptyset$ 
38:   Initialize  $foundCyclesNr \leftarrow 0$ 
39:   Initialize  $tempLength \leftarrow 0$ 
40:   Initialize  $truncated \leftarrow \text{False}$ 
41:   Call  $CyclesFromVertex(G, v, maxCycle, tempCycle, maxLength, tempLength, foundCycleNr, truncated, maxCorrections)$ 
42: end for
43:  $maxCycleCount \leftarrow maxCycleCount / \text{length of } maxPath$ 
44: return  $maxCycleCount, maxLength$  16
```

3.3 Tests

3.3.1 Test 1: Tree

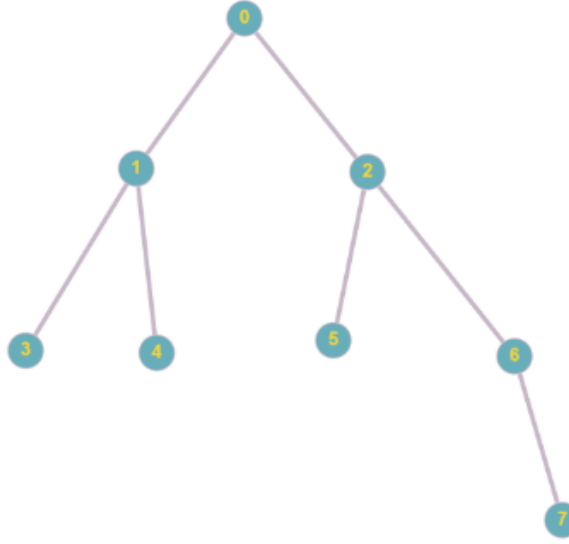


Figure 9: An undirected tree.

Execution Time (ms)	Length of the longest cycle
2	0

Table 1: Exact algorithm

Execution Time (ms)	Length of the longest cycle
2	0

Table 2: Approximated algorithm with maxCorrections=100

3.3.2 Test 2: Graph with many cycles

Execution Time (ms)	Length of the longest cycle
3965	21

Table 3: Exact algorithm

maxCorrections	Execution Time (ms)	Length of the longest cycle
10	7	16
100	30	21

Table 4: Approximated algorithm

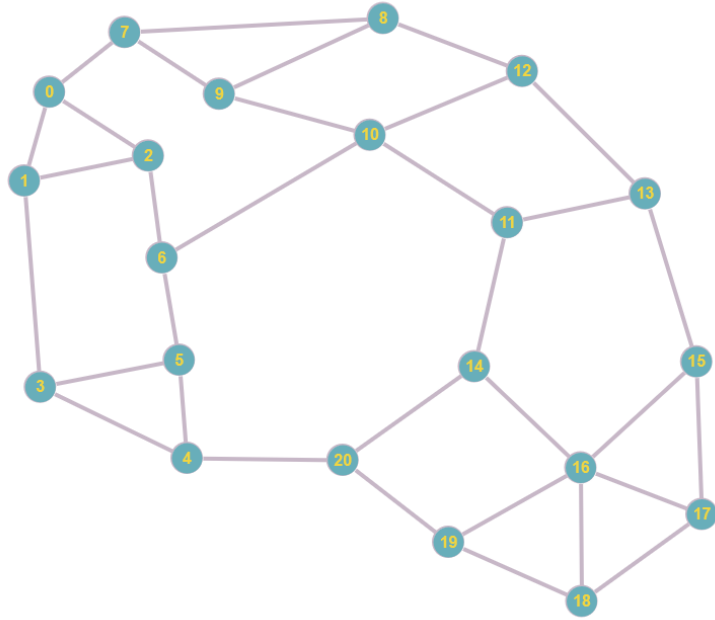


Figure 10: Graph with many cycles

3.3.3 Test 3: Very large, complex graph

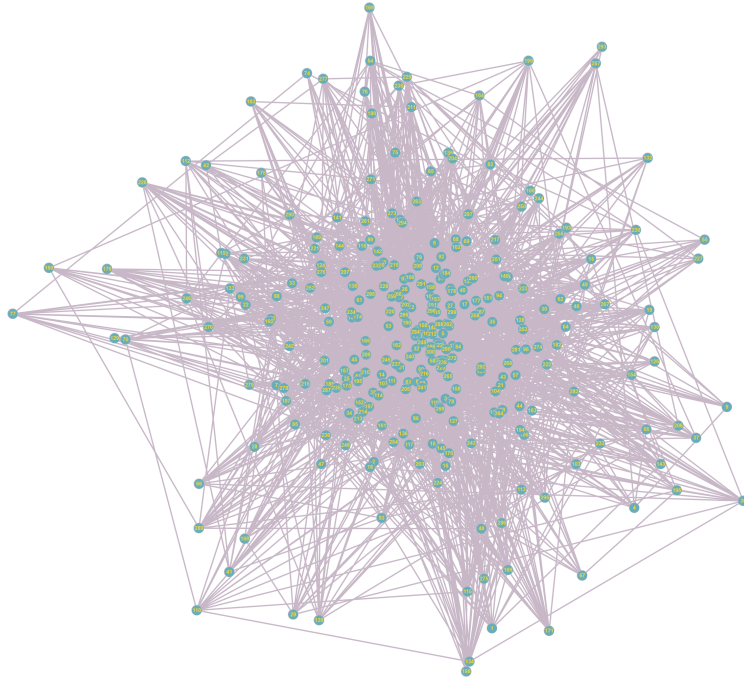


Figure 11: Very large, complex graph (300 vertices)

maxCorrections	Execution Time (ms)	Length of the longest cycle
1	6761	8
10	20505	51

Table 5: Approximated algorithm

3.3.4 Test 4: Multigraph

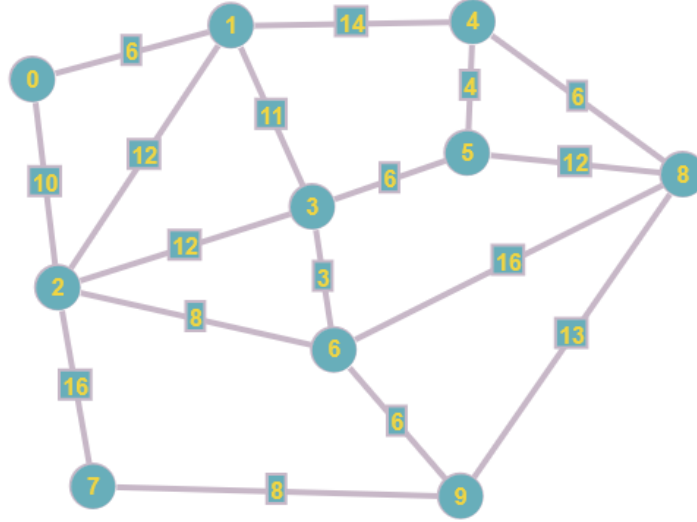


Figure 12: Very large, complex graph (300 vertices)

maxCorrections	Execution Time (ms)	Length
10	2	87
100	3	96
1000	17	99
10000	12	99

Table 6: Approximated algorithm

The example above illustrates, how the accuracy of the algorithm grows when we increase *maxCorrections*.

4 Minimum graph extension to ensure there is Hamiltonian Cycle in a graph

A **walk** is a finite or infinite sequence of edges which joins a sequence of vertices.

A **trail** is a walk in which all edges are distinct.

A **cycle** is a non-empty trail in which only the first and last vertices are equal.

A **Hamiltonian cycle** is a cycle that visits each vertex in a graph exactly once.

4.1 Exact algorithm

The algorithm for determining the exact extension to guarantee a Hamiltonian cycle in a graph is based on computing all Hamiltonian cycles in a complete graph of size $n = |V(G)|$ and identifying the cycle that shares the most edges with the given graph. This selected cycle is used to calculate the minimal extension needed. When multiple cycles have the same size intersection with the graph, the algorithm selects one at random and computes the difference between the sets of edges of the chosen cycle and the graph. The size of this difference represents the minimum number of edges that must be added to the graph to ensure it contains a Hamiltonian cycle.

The complexity of finding all permutations of an array is $O(n!)$. Then we create a Graph structure for each list permutation, by iterating through all of them. We iterate once more to check how many overlapping edges each cycle has with a given graph. Checking if an edge exists in a graph is done in constant time, as edges are stored in a set. The set difference is done in $O(n)$, because it just needs to iterate through the first collection and then check if the given element exists in the second set, which is done in constant time for a hash set. Overall, the complexity is $O(n!)$, because $n!$ is dominant term.

Algorithm 5 Generate All Hamiltonian Cycles of a Complete Graph

```
1: Input: List of integers vertices
2: Output: List of Graphs cycles
3: Initialize  $res \leftarrow \emptyset$  {Store all permutations of a list}
4: Define recursive function Permute(curr):
5:   if length of curr = length of vertices then
6:     Add a copy of curr to res
7:     return
8:   end if
9:   for each num in vertices do
10:    if  $num \notin curr$  then
11:      Append num to curr
12:      Permute(curr)
13:      Remove the last element of curr
14:    end if
15:  end for
16: Call Permute( $\emptyset$ )
17: Initialize cycles  $\leftarrow \emptyset$ 
18: for each cycle in res do
19:   Add Graph(cycle) to cycles
20: end for
21: Return cycles
```

Algorithm 6 Find Minimum Graph Extension to Ensure a Hamiltonian Cycle

```
1: Input: Graph  $graph$ 
2: Output: Minimum number of edges to add to make  $graph$  Hamiltonian
3:  $size \leftarrow$  number of vertices in  $graph$ 
4:  $cycles \leftarrow$  Algorithm 1( $\{0, 1, \dots, size - 1\}$ )
5: Initialize  $commonEdges \leftarrow \emptyset$  {Map each Hamiltonian cycle to its size of intersection
   with graph.}
6: Initialize  $maxCommonEdges \leftarrow 0$  {Maximum intersection size among all Hamiltonian
   cycles.}
7: for each  $cycle$  in  $cycles$  do
8:   Initialize  $commonEdgesCount \leftarrow 0$ 
9:   for each  $edge$  in  $cycle.edges$  do
10:    if  $edge \in graph.edges$  then
11:       $commonEdgesCount \leftarrow commonEdgesCount + 1$ 
12:    end if
13:  end for
14:   $commonEdges[cycle] \leftarrow commonEdgesCount$ 
15:  if  $commonEdgesCount > maxCommonEdges$  then
16:     $maxCommonEdges \leftarrow commonEdgesCount$ 
17:  end if
18: end for
19:  $hamiltonianCyclesWithBiggestIntersection \leftarrow$  all cycles in
    $commonEdges$  where value equals  $maxCommonEdges$ 
20: if  $hamiltonianCyclesWithMostMatching[0].edges \setminus graph.edges = \emptyset$  then
21:   Print "Graph already has Hamiltonian cycle"
22: else
23:    $edgesToAdd \leftarrow hamiltonianCyclesWithMostMatching[0].edges \setminus graph.edges$ 
24: end if
25: return  $|edgesToAdd|$ 
```

4.2 Proposed approximation algorithm

The approximation algorithm for this problem is based on the idea of finding the maximum path in a graph, deleting it from the original graph, and then finding another maximum path in the remaining graph. This process continues until the graph is empty. The longest paths found are stored in a list, and then the last element of each path is connected to the first element of the next path.

The algorithm for finding the maximum path, **Algorithm 7**, has exponential complexity, because in the worst case the algorithm has to check for all possible path combinations. Therefore, **Algorithm 9** was introduced to bound the search space using the *max_path* parameter. This parameter ensures that if no improvement is made within the last *max_path* iterations, the algorithm considers the longest path found so far as the final longest path. The algorithm was inspired by [2] and adjusted to find the longest path.

The **Algorithm 11**, which is responsible for finding the actual extension of the graph, identifies the longest path in a graph, removes it, and repeats this process on the remaining graph until the graph is empty. The longest paths are stored in a list, which is passed as an output parameter (a **C#** feature) and modified for use in an outer scope. The size of this list is returned, representing the number of connecting edges required to form a Hamiltonian cycle. It recognizes if the graph already contains a Hamiltonian cycle. If the algorithm finds the longest path and this path includes all the vertices, an appropriate message is printed.

Algorithm 7 InducedPathsFromVertex

```

1: Input: Graph  $G = (V, E)$ , vertex  $v$ , list maxPath, list tempPath
2: Output: Longest induced path from vertex  $v$  is added to maxPath
3: Increment exactExecutionCount
4: Add  $v$  to tempPath
5:  $neighbors \leftarrow G.GetNeighbors(v)$ 
6: Remove  $v$  from neighbors
7: if neighbors is not empty then
8:   for all  $neighbor \in neighbors$  do
9:      $remainingVertices \leftarrow V \setminus \{v\}$ 
10:     $remainingEdges \leftarrow \{(u, w) \in E : u, w \in remainingVertices\}$ 
11:     $subGraph \leftarrow Graph(remainingVertices, remainingEdges)$ 
12:    Call InducedPathsFromVertex(subGraph, neighbor, maxPath, tempPath)
13:   end for
14: else
15:   if tempPath.size() > maxPath.size() then
16:     maxPath.clear()
17:     maxPath.addAll(tempPath)
18:   end if
19: end if
20: Remove the last element of tempPath

```

Algorithm 8 ExactMaximumPath

```
1: Input: Graph  $G = (V, E)$ 
2: Output: The longest induced path in  $G$ 
3: Initialize  $maxPath \leftarrow \emptyset$ 
4: for all  $v \in V$  do
5:   Initialize  $tempPath \leftarrow \emptyset$ 
6:   Call InducedPathsFromVertex( $G, v, maxPath, tempPath$ )
7: end for
8: return  $maxPath$ 
```

Algorithm 9 FirstInducedPaths

```
1: Input: Graph  $G = (V, E)$ , vertex  $v$ , list  $maxPath$ , list  $tempPath$ , integer  $maxPaths$ ,
   integer  $paths$ , integer  $lastImprovement$ , boolean  $truncated$ 
2: Output: Compute paths, possibly truncating if the maximum paths without improve-
   ment is exceeded
3: Add  $v$  to  $tempPath$ 
4:  $neighbors \leftarrow G.GetNeighbors(v)$ 
5: Remove  $v$  from  $neighbors$ 
6: if  $neighbors$  is not empty then
7:   for all  $neighbor \in neighbors$  do
8:      $remainingVertices \leftarrow V \setminus \{v\}$ 
9:      $remainingEdges \leftarrow \{(u, w) \in E : u, w \in remainingVertices\}$ 
10:     $subGraph \leftarrow Graph(remainingVertices, remainingEdges)$ 
11:    Call FirstInducedPaths( $subGraph, neighbor, maxPath, tempPath, maxPaths,$ 
       $paths, lastImprovement, truncated$ )
12:    if  $truncated$  then
13:      return
14:    end if
15:  end for
16: else
17:   Increment  $paths$ 
18:   if  $tempPath.size() > maxPath.size()$  then
19:      $maxPath.clear()$ 
20:      $maxPath.addAll(tempPath)$ 
21:      $lastImprovement \leftarrow paths$ 
22:   end if
23:   if  $paths - lastImprovement > maxPaths$  then
24:      $truncated \leftarrow \text{true}$ 
25:     Clear  $tempPath$ 
26:   return
27: end if
28: end if
29: Remove the last element from  $tempPath$ 
```

Algorithm 10 HeuristicMaximumPath

```
1: Input: Graph  $G = (V, E)$ , integer  $maxPaths$ 
2: Output: Approximate longest path in  $G$ 
3: Initialize  $maxPath \leftarrow \emptyset$ 
4: for all  $v \in G.Vertices$  do
5:    $lastImprovement \leftarrow 0$ 
6:    $truncated \leftarrow \mathbf{false}$ 
7:    $paths \leftarrow 0$ 
8:   Initialize  $tempPath \leftarrow \emptyset$ 
9:   Call FirstInducedPaths( $G, v, maxPath, tempPath, maxPaths, paths,$ 
      $lastImprovement, truncated$ )
10: end for
11: return  $maxPath$ 
```

Algorithm 11 FindExtensionUsingHeuristicMaxPath

```
1: Input: Graph  $G = (V, E)$ , integer  $maxPaths$ 
2: Output: Set of approximate longest paths in  $G$  and their count
3: Initialize  $longestPaths \leftarrow \emptyset$ 
4: Initialize  $remainingGraph \leftarrow G$ 
5: while  $remainingGraph.Vertices \neq \emptyset$  do
6:    $longestPath \leftarrow \mathbf{Call}$  HeuristicMaximumPath( $remainingGraph, maxPaths$ )
7:   Add  $longestPath$  to  $longestPaths$ 
8:    $remainingGraph \leftarrow \mathbf{Call}$  RemovePathFromGraph( $remainingGraph, longestPath$ )
9: end while
10: if  $|longestPaths[0]| = |G.Vertices|$  and  $(longestPaths[0].Last, longestPaths[0].First) \in$ 
     $G.Edges$  then
11:   Print "Graph already has Hamiltonian cycle"
12:    $heuristicExecutionCount \leftarrow 0$ 
13:   return 0
14: else
15:   return  $|longestPaths|$ 
16: end if
```

4.3 Comparison of exact algorithm and approximation algorithm

4.3.1 Test 1

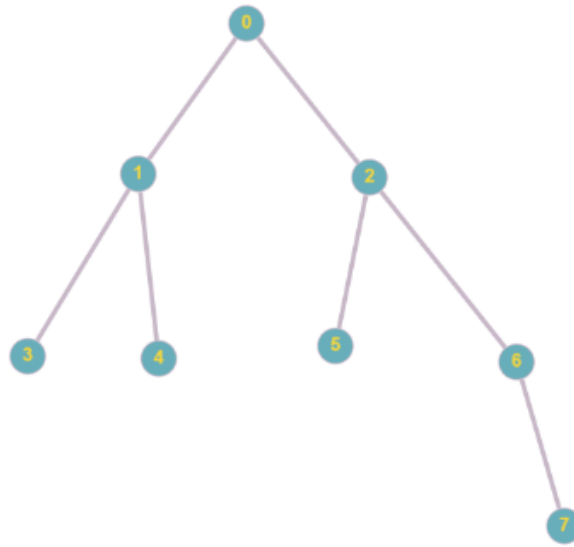


Figure 13: An undirected tree.

Execution Time (ms)	Number of Edges Added
286	3

Table 7: Exact algorithm

Algorithm suggests adding edges: (3,4), (4,5), (7,0)

Execution Time (ms)	Number of Edges Added
17	3

Table 8: Approximation algorithm max_paths=1

Algorithm suggests adding edges: (7, 4), (4, 5), (5, 3)

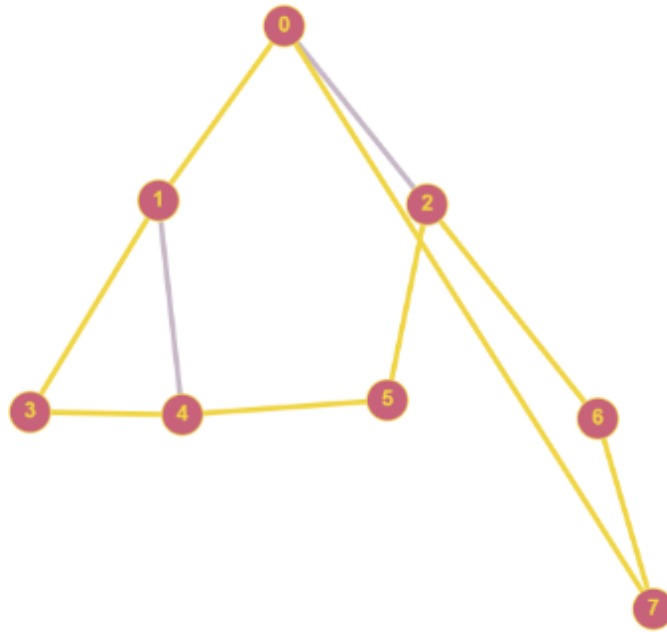


Figure 14: Hamiltonian cycle on extended graph using exact algorithm.

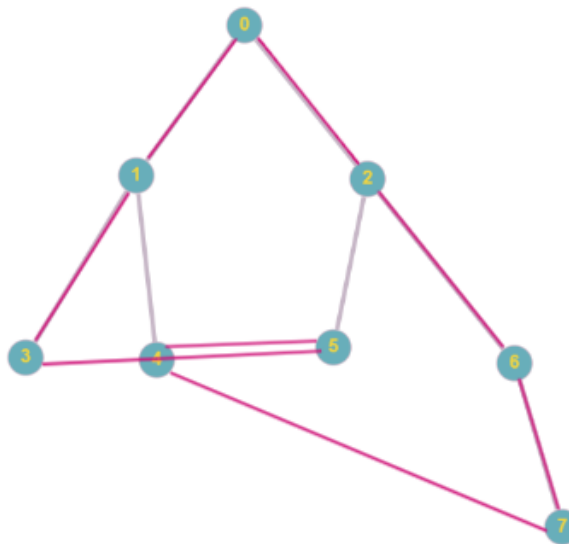


Figure 15: Hamiltonian cycle on extended graph using approximation algorithm.

4.3.2 Test 2

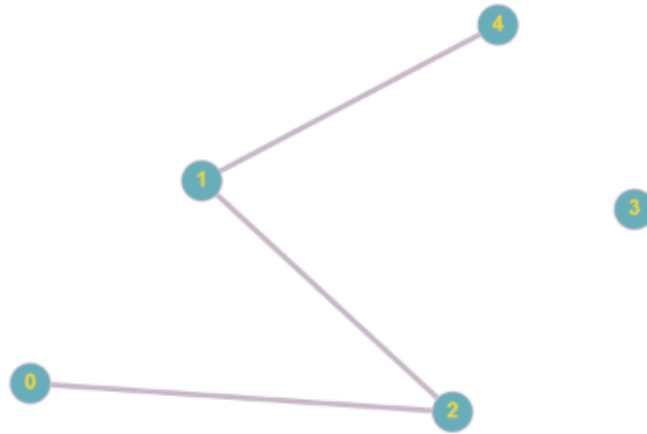


Figure 16: An undirected, disconnected graph.

Execution Time (ms)	Number of Edges Added
1	2

Table 9: Exact algorithm.

Algorithm suggests adding edges: (4, 3), (3, 0)

Execution Time (ms)	Number of Edges Added
0	2

Table 10: Approximation algorithm. max_paths=1

Algorithm suggests adding edges: (4, 3), (3, 0)

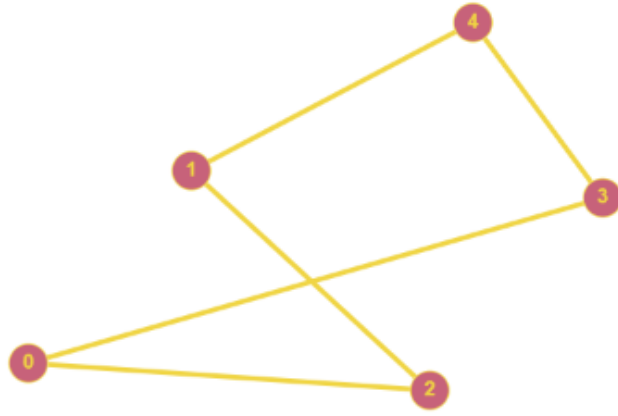


Figure 17: Hamiltonian cycle on extended graph.

4.3.3 Test 3

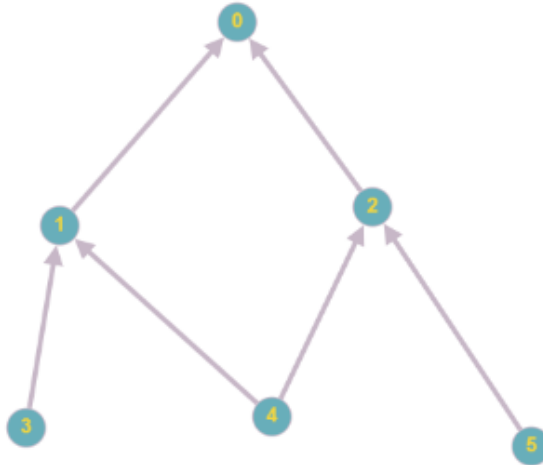


Figure 18: A directed connected graph.

Execution Time (ms)	Number of Edges Added
4	3

Table 11: Exact algoritm.

Algorithm suggests adding edges: (0, 3), (1, 4), (4, 5)

Execution Time (ms)	Number of Edges Added
2	3

Table 12: Approximation algorithm max_paths=1.

Algorithm suggests adding edges: (0, 4), (2, 5), (5, 3)

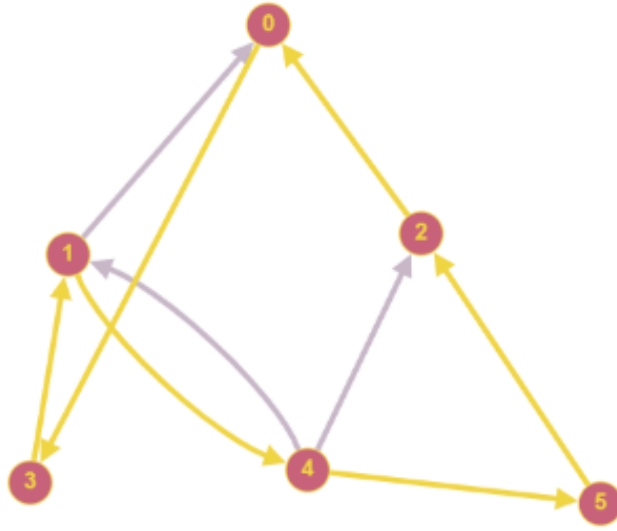


Figure 19: Hamiltonian cycle on extended graph using exact algorithm.

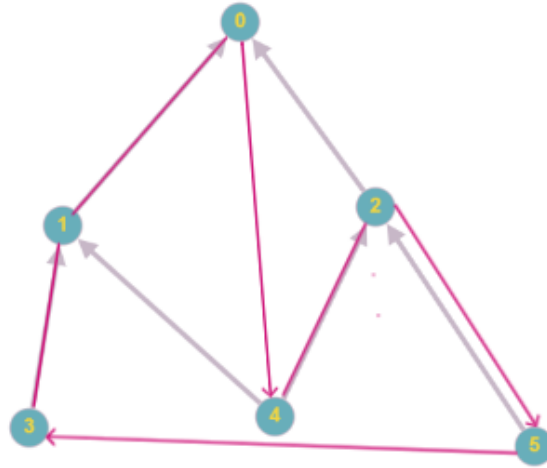


Figure 20: Hamiltonian cycle on extended graph using approximation algorithm.

4.3.4 Test 4

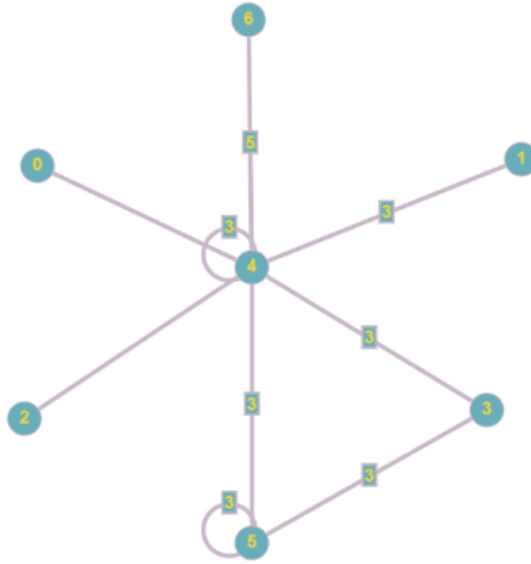


Figure 21: An undirected connected multigraph graph with self loops.

Execution Time (ms)	Number of Edges Added
33	4

Table 13: Exact algorithm.

Algorithm suggests adding edges: (0, 1), (1, 2), (2, 3), (6, 0)

Execution Time (ms)	Number of Edges Added
0	4

Table 14: Approximation algorithm $\text{math_paths} = 1$.

Algorithm suggests adding edges: (5, 1), (1, 2), (2, 6), (6, 0)

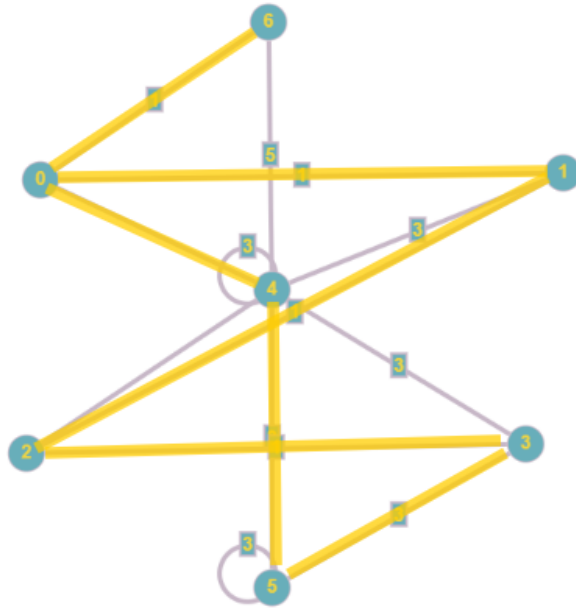


Figure 22: Hamiltonian cycle on extended graph using exact algorithm.

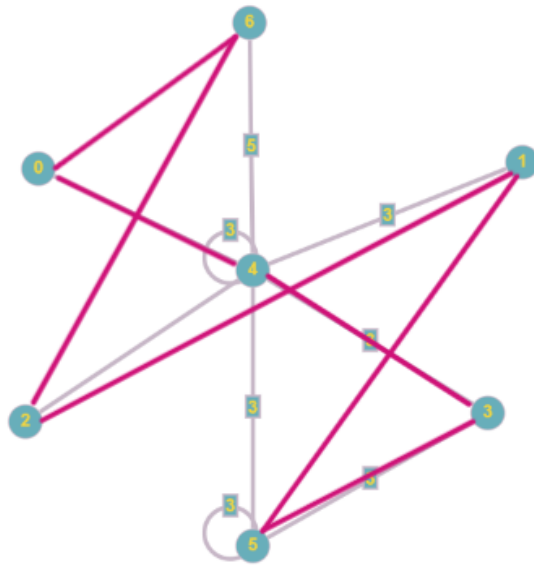


Figure 23: Hamiltonian cycle on extended graph using approximation algorithm.

4.3.5 Test 5



Figure 24: A directed connected multigraph graph.

Execution Time (ms)	Number of Edges Added
2423	6

Table 15: Exact algorithm.

Algorithm suggests adding edges: (0, 1), (2, 4), (4, 5), (5, 6), (7, 8), (8, 0)

Execution Time (ms)	Number of Edges Added
0	6

Table 16: Approximation algorithm math_paths = 1.

Algorithm suggests adding edges: (0, 1), (2, 4), (4, 5), (5, 7), (7, 8), (8, 6)

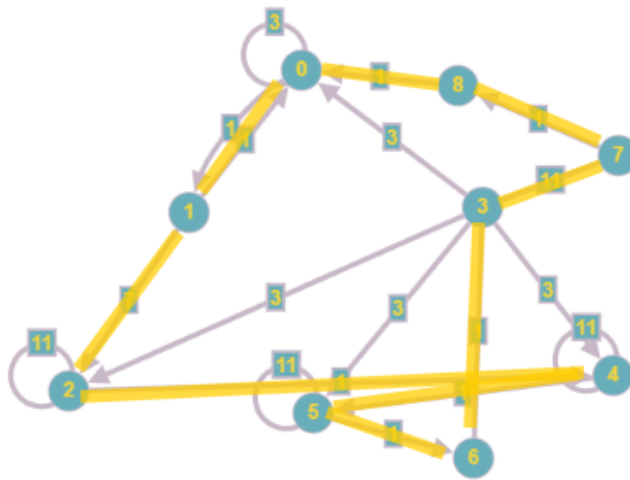


Figure 25: Hamiltonian cycle on extended graph using exact algorithm.

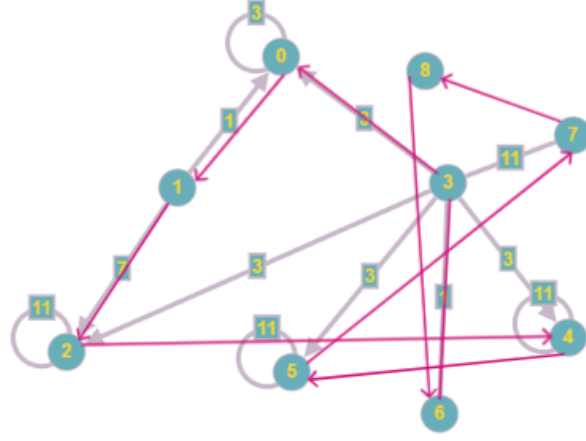


Figure 26: Hamiltonian cycle on extended graph using approximation algorithm.

4.4 Tests using approximation

4.4.1 Test run on graph of size 909 vertices

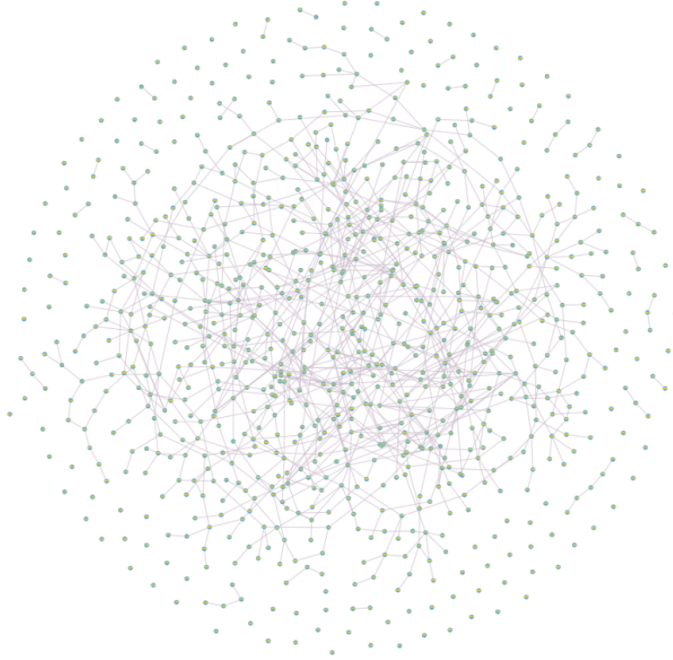


Figure 27: Sparse graph with 909 vertices.

Execution Time (ms)	Number of Edges Added
6904	376

Table 17: Execution time and number of edges added, max paths=10

Execution Time (ms)	Number of Edges Added
12971	376

Table 18: Execution time and number of edges added, max paths=20

Execution Time (ms)	Number of Edges Added
3749	375

Table 19: Execution time and number of edges added, max paths=5

By manipulating *max_paths* parameter, parameter we can get better accuracy of the result if we enlarge it, yet it results in longer execution time. The perfect size of *max_paths* depends on the graph structure and size. For each case, a different value may be more appropriate.



Figure 28: Sparse graph with 909 vertices structure

4.4.2 Test run on random undirected sparse graphs

Execution count refers to how many times a recursive **Algorithm 6** function was called.

Number of Vertices	Execution Time (ms)	Number of Edges Added
10	1	4
20	1	13
30	1	20
40	4	25
50	0	48
60	93	13
70	1	61
80	7	63
90	275	29
100	284	41
110	265	45
120	476	46
130	21	86
140	93	76

Table 20: Tests run on random sparse undirected graphs with max_paths=5.

Number of Vertices	Execution Time (ms)	Number of Edges Added
10	0	4
20	0	13
30	0	20
40	1	25
50	0	48
60	56	13
70	1	61
80	1	63
90	203	29
100	190	41
110	240	45
120	412	46
130	13	86
140	85	76

Table 21: Tests run on random sparse undirected graphs with max_paths=1.

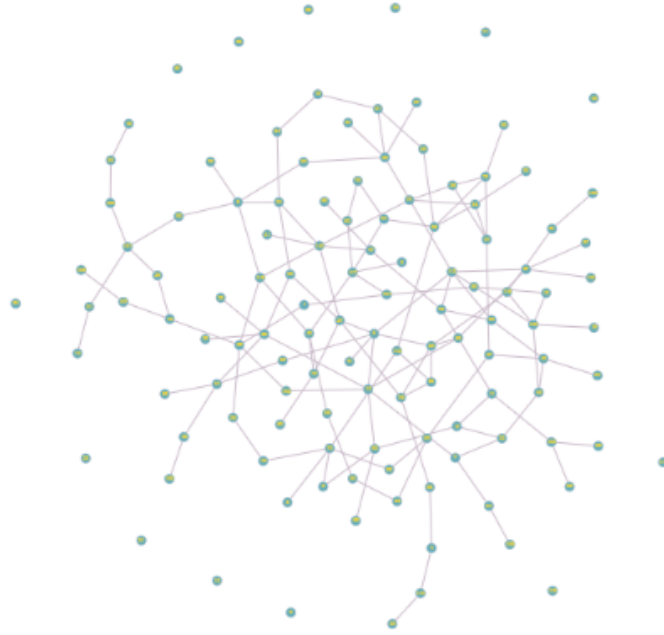


Figure 29: Sparse graph with 120 vertices structure from above dataset

Number of Vertices	Execution Time (ms)	Number of Edges Added
10	2	1
20	38	2
30	23	3
40	68	4
50	407	4
60	642	3
70	1352	4
80	2222	2
90	2552	3

Table 22: Tests run on random directed graphs with max_paths=1.

Number of Vertices	Execution Time (ms)	Number of Edges Added
10	2	1
20	30	2
30	106	3
40	73	4
50	208	4
60	393	3
70	903	4
80	1630	2
90	3383	3

Table 23: Tests run on random directed graphs with $\text{max_paths}=5$.

4.4.3 Complexity analysis

In this analysis, We assume that the code is run on a dense graph in which there is high probability that Hamiltonian Cycle already exists. The algorithm quickly finds the longest path in such a dense graph and makes effort to enhance this result max_path times.

In this test, the max_path parameter is increased in every iteration. The number of iterations of the recursive inner function *FirstInducedPaths* grows linearly with the increase of the parameter. The test was run on a dense graph to ensure there are as many redundant operations as possible to test the execution count in an extreme case.

4.4.4 Influence of max paths parameter on the complexity

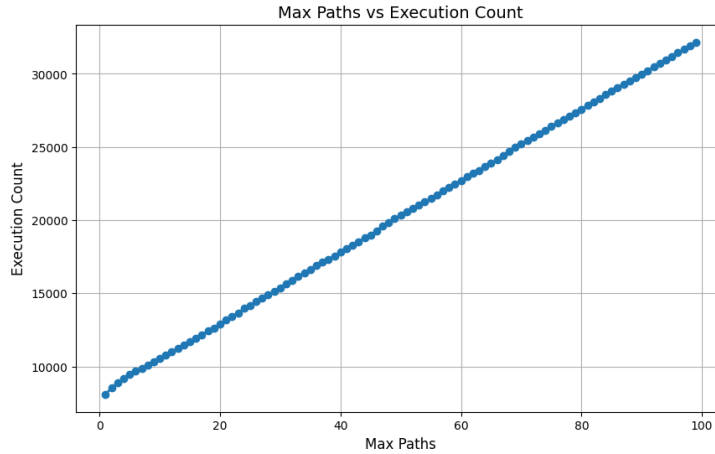


Figure 30: Relationship between max paths parameter and execution count on dense graph

In **Algorithm 11**, the outer loop will execute at most n times for a graph consisting of disconnected vertices. This is because the longest path identified in each iteration has a length of one, requiring the removal of each vertex individually from the graph. The best-case scenario occurs with complete graphs, as the longest path in such graphs includes all vertices. Consequently, all vertices are removed in a single iteration, causing the loop to execute only once.

Within **Algorithm 10**, the **HeuristicMaximumPath** function is invoked, containing a loop that iterates over the n vertices of the graph. This function, in turn, calls **FirstInducedPaths**, which includes a loop that executes up to $n - 1$ times if a vertex is connected to all other vertices in the graph. The best-case scenario arises for a set of disconnected vertices, where none of the vertices have neighbors, resulting in minimal iterations. The **FirstInducedPaths** calls itself recursively. The search space is bounded by *max_paths* parameter and for that reason if the longest path found is not improved for *max_paths* iterations then the recursive function returns.

The time complexity of this algorithm is $O(n^2 * k)$, where $k = \text{max_paths}$ because finding the longest path takes n iterations and for each time the new longest path is found the algorithm can try to improve it k times.

References

- [1] Miquel Ferrer, Francesc Serratosa, and Kaspar Riesen. “A First Step Towards Exact Graph Edit Distance Using Bipartite Graph Matching”. In: *Graph-Based Representations in Pattern Recognition*. Ed. by Cheng-Lin Liu et al. Cham: Springer International Publishing, 2015, pp. 77–86. ISBN: 978-3-319-18224-7.
- [2] Marzo, Ruslán G. and Ribeiro, Celso C. “Exact and approximate algorithms for the longest induced path problem”. In: *RAIRO-Oper. Res.* 55.2 (2021), pp. 333–353. DOI: 10.1051/ro/2021004. URL: <https://doi.org/10.1051/ro/2021004>.
- [3] Robert Pilgrim. *Tutorial on Implementation of Munkres’ Assignment Algorithm*. Aug. 1995. DOI: 10.13140/RG.2.1.3572.3287.
- [4] Kaspar Riesen. *Structural Pattern Recognition with Graph Edit Distance: Approximation Algorithms and Applications*. eng. 1st ed. 2015. Advances in Computer Vision and Pattern Recognition. Cham: Springer Nature, 2016. ISBN: 3319272527.